



Funções Hash

Prof. Dr. Iaçanã Ianiski Weber

Confiabilidade e Segurança de Software

98G08-4

Agradecimentos especiais ao Prof. Avelino Zorzo e aos Autores Christof Paar e Jan Pelzl pelo material.

1 Porque Precisamos de Funções Hash

2 Propriedades de Funções de Hash

3 SHA-2

4 SHA-3

1 Porque Precisamos de Funções Hash

2 Propriedades de Funções de Hash

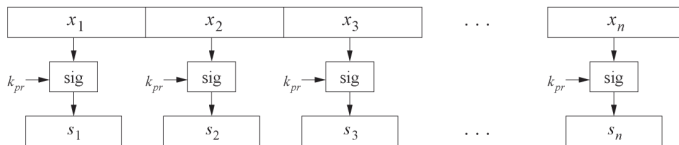
3 SHA-2

4 SHA-3

Por que usar Funções de Hash para Assinaturas?

O Problema: Assinatura ingênua de mensagens longas

Assinar uma mensagem longa em blocos separados gera uma assinatura de tamanho similar ao da própria mensagem, o que é ineficiente.



Isto gera três problemas principais:

- **Sobrecarga Computacional:** Operações de chave pública são lentas e custosas para cada bloco.
- **Sobrecarga de Mensagem:** A assinatura final fica muito grande.
- **Limitações de Segurança:** Um atacante pode reordenar, remover ou duplicar blocos para forjar novas mensagens.

Solução

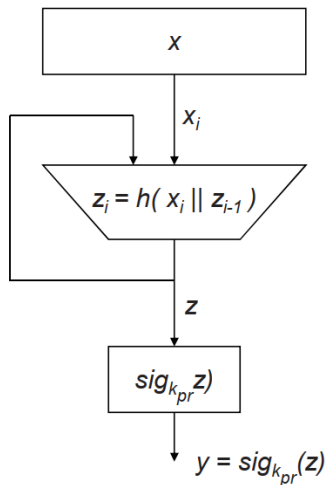
Em vez de assinar a mensagem inteira, assina-se apenas seu **digest** (ou **hash**).

→ Igualmente seguro, muito mais rápido.

Necessidade

Isso requer o uso de **Funções de Hash** criptográficas.

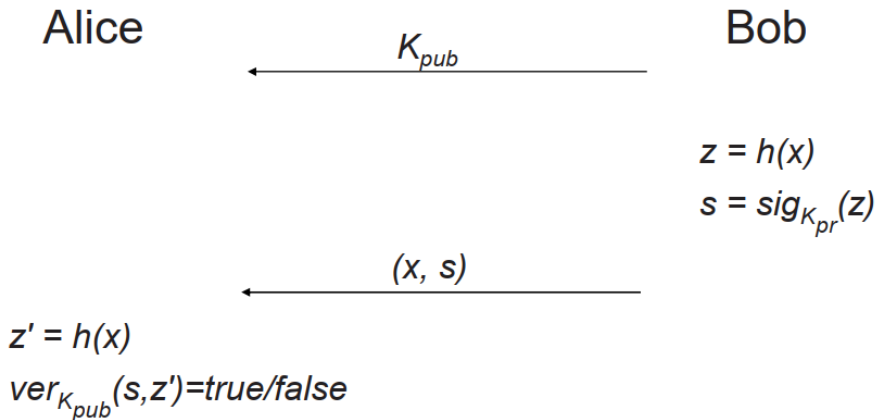
Assinatura Digital com Função Hash



Notes:

- x has fixed length
- z , y have fixed length
- z , x do not have equal length in general
- $h(x)$ does not require a key.
- $h(x)$ is public.

Protocolo Básico de Assinatura Digital usando Função Hash



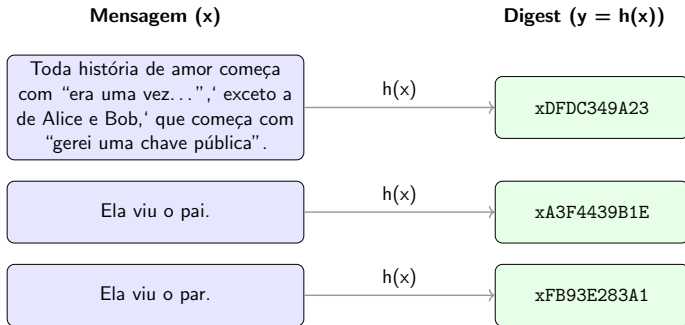
1 Porque Precisamos de Funções Hash

2 Propriedades de Funções de Hash

3 SHA-2

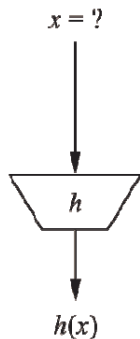
4 SHA-3

O que uma Função de Hash faz?

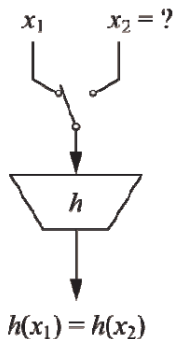


- **Entrada de tamanho arbitrário:** A função h opera sobre uma mensagem x de qualquer comprimento.
- **Saída de tamanho fixo:** O resultado (digest) $y = h(x)$ tem sempre o mesmo comprimento (ex: 256 bits).
- **Função de mão única (one-way):** Dado x , é fácil computar $h(x)$. Dado $h(x)$, é computacionalmente inviável encontrar x .

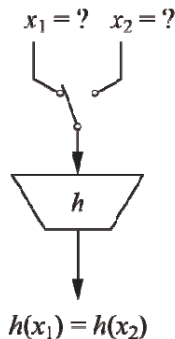
As três propriedades das Funções Hash



preimage resistance



second preimage
resistance



collision resistance

As Três Garantias Fundamentais

- **Resistência à pré-imagem (Preimage resistance)**

Para uma dada saída z , é computacionalmente inviável encontrar qualquer entrada x tal que $h(x) = z$.

- Essencialmente, a propriedade de ser uma função de **mão única** (*one-way*).

- **Resistência à segunda pré-imagem (Second preimage resistance)**

Dado um bloco de dados x_1 , é computacionalmente inviável encontrar *outro* bloco x_2 tal que $x_1 \neq x_2$ e que $h(x_1) = h(x_2)$.

- Protege contra a falsificação de um documento já assinado.

- **Resistência à colisão (Collision resistance)**

É computacionalmente inviável encontrar *qualquer par* de entradas distintas x_1, x_2 tal que $x_1 \neq x_2$ e que $h(x_1) = h(x_2)$.

- É uma propriedade mais forte que a resistência à segunda pré-imagem.

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.
- **Pergunta análoga:** Quantas pessoas são necessárias em uma sala para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja de 50%?

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.
- **Pergunta análoga:** Quantas pessoas são necessárias em uma sala para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja de 50%?
 - A resposta intuitiva seria $365/2 \approx 183$

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.
- **Pergunta análoga:** Quantas pessoas são necessárias em uma sala para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja de 50%?
 - A resposta intuitiva seria $365/2 \approx 183$? **Não!**
- Apenas **23 pessoas** são suficientes! Isso é conhecido como o **Paradoxo do Aniversário**.

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.
- **Pergunta análoga:** Quantas pessoas são necessárias em uma sala para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja de 50%?
 - A resposta intuitiva seria $365/2 \approx 183$? **Não!**
- Apenas **23 pessoas** são suficientes! Isso é conhecido como o **Paradoxo do Aniversário**.
- **Implicação para Hashes:** Um ataque para encontrar colisões (chamado de *birthday attack*) é muito mais eficiente do que um ataque de força bruta. A busca leva aproximadamente $2^{n/2}$ passos para uma função de hash com saída de n bits.

O Paradoxo do Aniversário

- A **resistência à colisão** é a propriedade que, na prática, causa mais problemas de segurança.
- **Pergunta análoga:** Quantas pessoas são necessárias em uma sala para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja de 50%?
 - A resposta intuitiva seria $365/2 \approx 183$? **Não!**
- Apenas **23 pessoas** são suficientes! Isso é conhecido como o **Paradoxo do Aniversário**.
- **Implicação para Hashes:** Um ataque para encontrar colisões (chamado de *birthday attack*) é muito mais eficiente do que um ataque de força bruta. A busca leva aproximadamente $2^{n/2}$ passos para uma função de hash com saída de n bits.

O Paradoxo do Aniversário

Por que o paradoxo funciona?

- A intuição falha porque não consideramos o número de **pares** possíveis. Com apenas 23 pessoas, existem 253 pares distintos, aumentando drasticamente a chance de uma coincidência.
- Em um ataque, o atacante não precisa quebrar o hash de um documento específico. Ele pode gerar várias versões de um documento bom e de um malicioso até que *qualquer par* entre eles produza o mesmo hash, o que é um problema muito mais fácil de resolver.

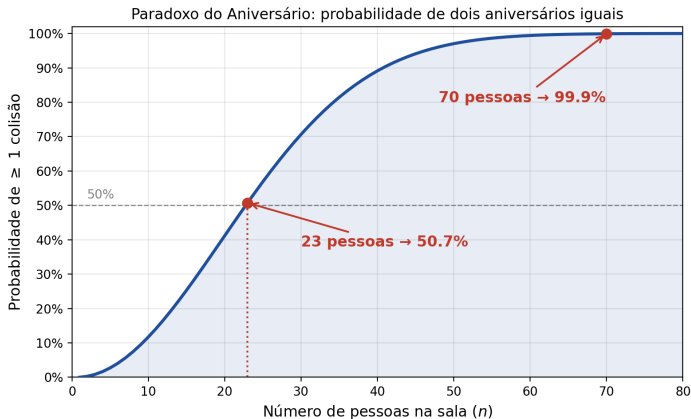
O Paradoxo do Aniversário

Por que o paradoxo funciona?

- A intuição falha porque não consideramos o número de **pares** possíveis. Com apenas 23 pessoas, existem 253 pares distintos, aumentando drasticamente a chance de uma coincidência.
- Em um ataque, o atacante não precisa quebrar o hash de um documento específico. Ele pode gerar várias versões de um documento bom e de um malicioso até que *qualquer par* entre eles produza o mesmo hash, o que é um problema muito mais fácil de resolver.

Recomendação de Segurança: Para se proteger deste ataque, a recomendação é que a saída de uma função de hash deve ter **no mínimo 224 bits**.

O Paradoxo do Aniversário em Números



- A curva sobe muito mais rápido do que a intuição sugere: com 23 pessoas ($\approx 6\%$ dos 365 dias!) a colisão já é mais provável que não, pois o número de **pares** cresce como $\binom{n}{2}$.
- Para funções de hash vale o mesmo: com $\approx \sqrt{2^n} = 2^{n/2}$ tentativas, encontrar *alguma* colisão torna-se provável: é o *birthday attack*.

O Ataque de Aniversário e a Segurança Moderna

- O **Paradoxo do Aniversário** mostra que é muito mais fácil encontrar *qualquer* colisão do que uma específica.
 - Para um hash de n bits, a complexidade de um ataque de aniversário é de aproximadamente $2^{n/2}$ operações.
- O **Padrão Antigo (Obsoleto)**
 - A recomendação de 160 bits (do algoritmo SHA-1) oferecia $160/2 = 80$ bits de segurança contra colisão.
 - Este nível de segurança (2^{80}) **não é mais considerado seguro** contra adversários com recursos significativos.
- O **Recomendação Atual (NIST)**
 - O NIST exige um nível de segurança de no mínimo **112 bits** contra colisão.
 - Para isso, a saída do hash deve ter no mínimo $2 \times 112 = \mathbf{224 \text{ bits}}$.
- **Conclusão Prática para Novos Sistemas:**

O Ataque de Aniversário e a Segurança Moderna

- O **Paradoxo do Aniversário** mostra que é muito mais fácil encontrar *qualquer* colisão do que uma específica.
 - Para um hash de n bits, a complexidade de um ataque de aniversário é de aproximadamente $2^{n/2}$ operações.
- O **Padrão Antigo (Obsoleto)**
 - A recomendação de 160 bits (do algoritmo SHA-1) oferecia $160/2 = 80$ bits de segurança contra colisão.
 - Este nível de segurança (2^{80}) **não é mais considerado seguro** contra adversários com recursos significativos.
- A **Recomendação Atual (NIST)**
 - O NIST exige um nível de segurança de no mínimo **112 bits** contra colisão.
 - Para isso, a saída do hash deve ter no mínimo $2 \times 112 = \mathbf{224 \text{ bits}}$.
- **Conclusão Prática para Novos Sistemas:**
 - Use no mínimo **SHA-256** (saída de 256 bits).
 - Abandone completamente o uso de SHA-1.

Funções de Hash: Lições Aprendidas

- **Natureza e Uso Principal**

- Funções de Hash não usam chaves (*keyless*).
- Suas aplicações mais importantes são em assinaturas digitais e em códigos de autenticação de mensagens (MACs), como o HMAC.

- **Requisitos de Segurança Fundamentais**

- As três propriedades de segurança essenciais são a resistência à pré-imagem (one-wayness), à segunda pré-imagem e à colisão.

- **Estado Atual dos Algoritmos**

- **MD5 e SHA-1:** São considerados **inseguros**. O SHA-1 possui falhas graves e seu uso deve ser completamente descontinuado.
- **SHA-2:** A família de algoritmos SHA-2 (ex: SHA-256, SHA-512) continua sendo considerada segura e é o padrão de mercado atual.
- **SHA-3 (Atualização):** A competição SHA-3 **terminou em 2012**. O algoritmo vencedor (Keccak) foi padronizado em 2015. Ele serve como uma alternativa moderna ao SHA-2.

- **Recomendação de Tamanho (Atualizada)**

- Para segurança de longo prazo e para resistir a ataques de colisão (ataques de aniversário), a recomendação atual é usar saídas de no mínimo **256 bits**.

1 Porque Precisamos de Funções Hash

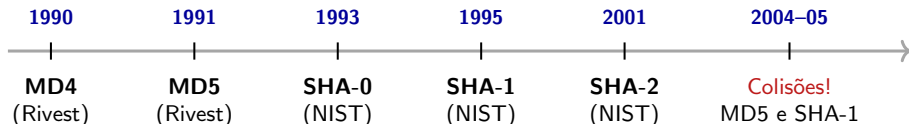
2 Propriedades de Funções de Hash

3 **SHA-2**

4 SHA-3

De MD4 ao SHA-2: Uma Breve História

SHA-2 não nasceu do zero: ele é o integrante mais bem-sucedido da família MD4.



- **A ideia do MD4** (Rivest, o “R” do RSA): ser **rápido em software**, usando apenas operações bit a bit sobre palavras de 32 bits (AND, OR, XOR, NOT). *Todos* os descendentes herdaram esse princípio.
- MD5 (128 bits) e SHA-1 (160 bits) pareciam seguros... até que **colisões reais** foram anunciadas em 2004–2005. Hoje, uma colisão de SHA-1 custa apenas $\approx 2^{61}$ operações (vs. os 2^{80} esperados).
- A resposta do NIST (2001): **mesma receita, saídas bem maiores**, com SHA-256, SHA-384 e SHA-512 (SHA-224 veio em 2004). Nascia a família **SHA-2**.
- **Moral da história:** SHA-2 é o “sobrevivente endurecido” da família MD4: mesmo DNA, mas com margens de segurança muito maiores.

A Família SHA-2: Seis Algoritmos

O nome diz tudo: o número após “SHA-” é o tamanho da saída em bits.

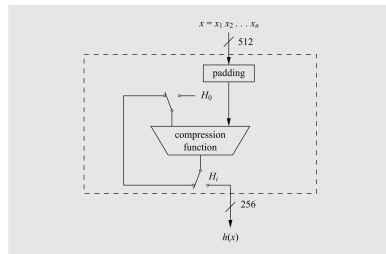
Algoritmo	Saída [bits]	Bloco [bits]	Rodadas	Colisões?
MD5	128	512	64	encontradas
SHA-1	160	512	80	encontradas
SHA-224	224	512	64	não
SHA-256	256	512	64	não
SHA-384	384	1024	80	não
SHA-512	512	1024	80	não

- Em verde: a família SHA-2. Há ainda duas variantes truncadas, **SHA-512/224** e **SHA-512/256**.
- A estrutura interna dos seis algoritmos é idêntica; por isso, nesta seção estudamos só o **SHA-256** e, de quebra, entendemos todos.
- Limites de mensagem: SHA-256 aceita até $2^{64} - 1$ bits; SHA-384/512, até $2^{128} - 1$ bits.

SHA-256: Visão de Alto Nível (Merkle–Damgård)

Como digerir uma mensagem de qualquer tamanho? Em fatias.

- A mensagem recebe um **padding** e é dividida em **blocos de 512 bits**.
- Um **estado de 256 bits** (H_i) atravessa a **função de compressão** bloco a bloco: o resultado de um bloco vira entrada para o próximo (construção **Merkle–Damgård**).
- Depois do último bloco, o estado final é o hash $h(x)$.
- **Insight:** a função de compressão age como uma **cifra de bloco disfarçada**, em que a “mensagem” cifrada é o estado H_{i-1} e a “chave” é o bloco x_i . As rodadas internas lembram uma cifra de Feistel.



Pré-processamento: o Padding

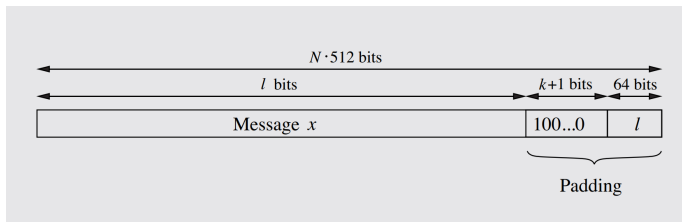
Objetivo: fazer o comprimento total ser um **múltiplo de 512 bits**.

A receita (para uma mensagem x de l bits):

- 1 Anexa-se um único bit 1;
- 2 Anexam-se k bits 0, onde k é o *menor* valor que fecha a conta:

$$k \equiv 448 - (l + 1) \pmod{512}$$

- 3 Anexa-se o comprimento l codificado em **64 bits**.



- **Por que 64 bits para o comprimento?** O SHA-256 aceita mensagens de até $2^{64} - 1$ bits, e $448 = 512 - 64$ reserva exatamente esse espaço no fim do último bloco.

Padding na Prática: a mensagem "abc"

Mensagem "abc" = 3 caracteres ASCII de 8 bits $\Rightarrow l = 24$ bits:

$\underbrace{01100001}_a \quad \underbrace{01100010}_b \quad \underbrace{01100011}_c$

Passo 1: quantos zeros?

$$k \equiv 448 - (l + 1) = 448 - 25 = 423 \pmod{512}$$

Passo 2: montar o bloco (comprimento $l = 24_{10} = 11000_2$ em 64 bits):

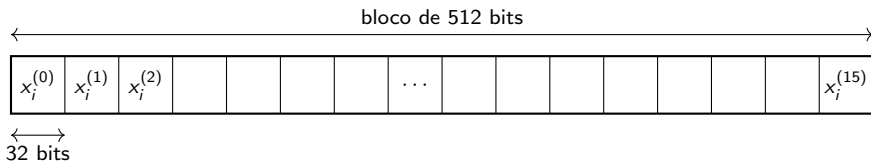
$\underbrace{01100001}_a \quad \underbrace{01100010}_b \quad \underbrace{01100011}_c \quad 1 \quad \underbrace{00 \cdots 0}_{423 \text{ zeros}} \quad \underbrace{0 \cdots 011000}_{l=24 \text{ em 64 bits}}$

Conferindo: $24 + 1 + 423 + 64 = 512$, ou seja, a mensagem ocupa exatamente **um** bloco.

Dividindo o Bloco em Palavras

Antes da compressão, cada bloco de 512 bits é visto como **16 palavras de 32 bits**:

$$x_i = \left(x_i^{(0)} \ x_i^{(1)} \ \dots \ x_i^{(15)} \right)$$



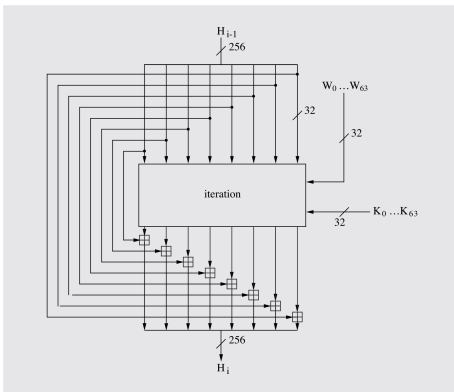
- **Por que 32 bits?** É o tamanho clássico dos registradores de CPU: *todas* as operações internas do SHA-256 (somas, XOR, rotações) atuam sobre palavras de 32 bits, herança direta do projeto “rápido em software” do MD4.

A Função de Compressão: Visão Geral

Para cada bloco x_i , a função de compressão executa **64 rodadas**.

Três ingredientes:

- O estado anterior H_{i-1} (256 bits = 8 palavras);
- O message schedule $W_0, \dots, W_{63}$: 64 palavras derivadas do próprio bloco x_i ;
- As constantes $K_0, \dots, K_{63}$: 64 palavras fixas do padrão.
- Em cada rodada j entram **uma** palavra W_j e **uma** constante K_j .
- O símbolo $\boxplus$ nos diagramas é a **soma módulo 2^{32}** , uma soma comum que “dá a volta” ao estourar 32 bits.



O Message Schedule W_j

Antes da fórmula, duas operações básicas (exemplo com 8 bits):

- $ROTR^n(x)$ é a **rotação** à direita: os bits dão a volta.
 $ROTR^3(10110010) = 01010110$
- $SHR^n(x)$ é o **deslocamento** à direita: os bits caem fora.
 $SHR^3(10110010) = 00010110$

A regra: as 16 primeiras palavras são *o próprio bloco*; as 48 seguintes misturam quatro palavras anteriores:

$$W_j = \begin{cases} x_i^{(j)} & 0 \leq j \leq 15 \\ \sigma_1(W_{j-2}) + W_{j-7} + \sigma_0(W_{j-15}) + W_{j-16} & 16 \leq j \leq 63 \end{cases}$$

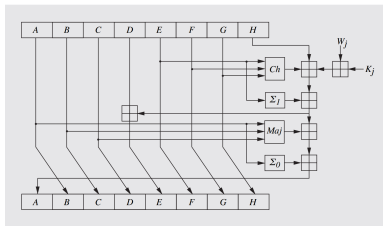
$$\sigma_0(x) = ROTR^7(x) \oplus ROTR^{18}(x) \oplus SHR^3(x)$$

$$\sigma_1(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x)$$

- **Efeito avalanche:** cada palavra nova depende de quatro anteriores; mudar *um único bit* do bloco contamina todo o schedule.

Uma Rodada por Dentro (iteração j)

O estado de 256 bits vive em **8 registradores** de 32 bits: $A, B, \dots, H$.



Em cada uma das 64 rodadas, calculam-se **duas palavras temporárias** e o resto **desliza**:

$$T_1 = H + \Sigma_1(E) + Ch(E, F, G) + K_j + W_j \quad T_2 = \Sigma_0(A) + Maj(A, B, C)$$

$$H=G, \quad G=F, \quad F=E, \quad E=D+T_1, \quad D=C, \quad C=B, \quad B=A, \quad A=T_1+T_2$$

- **Leitura didática:** só A e E recebem valores “novos” (somas módulo 2^{32}); os outros seis registradores apenas andam uma posição, como uma fila. E repare que há **só somas, XOR, AND e rotações**: por isso o SHA-2 é tão rápido em software.

As Funções Internas: *Ch*, *Maj* e Σ

- Na **Ch** (“choice”), x funciona como um **seletor** bit a bit:

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$

Se o bit de x é 1, sai o bit de y ; se é 0, sai o bit de z .

- A **Maj** (“majority”) é o **voto da maioria** bit a bit:

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

Cada bit de saída é o valor que aparecer em *pelo menos dois* dos três bits de entrada.

- Σ_0 e Σ_1 fazem a **difusão** por rotações sobrepostas:

$$\Sigma_0(x) = ROTR^2(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x)$$

$$\Sigma_1(x) = ROTR^6(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x)$$

Cada bit de saída mistura três bits distantes da entrada.

Exemplo rápido (1 bit): $x=1, y=0, z=1$: $Ch = y = 0$ e $Maj = 1$
(dois “votos” 1).

De Onde Vêm as Constantes? (*nothing up my sleeve*)

Pergunta natural: quem escolheu os 64 valores K_j e os 8 valores iniciais H_0 ? Não poderiam esconder um *backdoor*?

- **As constantes K_j :** os primeiros 32 bits da **parte fracionária das raízes cúbicas** dos 64 primeiros números primos ($\sqrt[3]{2}, \sqrt[3]{3}, \sqrt[3]{5}, \dots$).
- **Os valores iniciais H_0 :** idem, com as **raízes quadradas** dos 8 primeiros primos.
Exemplo:

$$\sqrt{2} = 1,414213562\dots \Rightarrow \lfloor 0,414213562\dots \times 2^{32} \rfloor = 0x6A09E667 = H_0^{(0)}$$

- São os chamados números **nothing up my sleeve** (“nada na manga”): derivados de constantes matemáticas **públicas e verificáveis**: qualquer pessoa pode refazer a conta.
- Isso demonstra que **não há valores especialmente construídos** que possam esconder uma fraqueza proposital no algoritmo.

Saída Final e Desempenho na Prática

O hash final: processados todos os n blocos, o digest é a concatenação das 8 palavras do estado:

$$\text{SHA-256}(x) = H_n^{(0)} \parallel H_n^{(1)} \parallel \dots \parallel H_n^{(7)} \quad (8 \times 32 = 256 \text{ bits})$$

- **Em software, SHA-2 voa:** só operações bit a bit e somas de 32 bits $\Rightarrow \approx 10$ Gbit/s em CPUs de 64 bits modernas.
- Vários processadores x86 (Intel e AMD) têm **instruções SHA dedicadas**, acelerando ainda mais.
- **Em hardware (FPGA), nem tanto:** poucos Gbit/s, pois as operações internas são difíceis de paralelizar; o desempenho fica na mesma faixa do software.

Saída Final e Desempenho na Prática

O hash final: processados todos os n blocos, o digest é a concatenação das 8 palavras do estado:

$$\text{SHA-256}(x) = H_n^{(0)} \parallel H_n^{(1)} \parallel \dots \parallel H_n^{(7)} \quad (8 \times 32 = 256 \text{ bits})$$

- **Em software, SHA-2 voa:** só operações bit a bit e somas de 32 bits $\Rightarrow \approx 10$ Gbit/s em CPUs de 64 bits modernas.
- Vários processadores x86 (Intel e AMD) têm **instruções SHA dedicadas**, acelerando ainda mais.
- **Em hardware (FPGA), nem tanto:** poucos Gbit/s, pois as operações internas são difíceis de paralelizar; o desempenho fica na mesma faixa do software.
- **Gancho para a próxima seção:** o **SHA-3** foi projetado com uma estrutura interna completamente diferente, e bem mais amigável a hardware.

- 1 Porque Precisamos de Funções Hash
- 2 Propriedades de Funções de Hash
- 3 SHA-2
- 4 **SHA-3**

SHA-3: Motivação e Contexto

- Em 2007, o NIST lança uma competição pública para definir uma nova função de hash: **SHA-3**.
- Motivação principal:
 - Ter uma alternativa a **SHA-2** com **projeto interno diferente**.
 - Garantir continuidade da segurança caso ocorra algum avanço criptanalítico contra o SHA-2.
- Diferentemente do AES (que substituiu o DES), **SHA-2 e SHA-3 foram projetados para coexistir**.
- O processo de seleção foi semelhante ao do AES:
 - Competição aberta.
 - Forte participação e escrutínio da comunidade científica.
- SHA-3 foi padronizado em 2015 como **FIPS 202**.

Linha do Tempo da Competição SHA-3

- **Nov 2007:** NIST anuncia a chamada de algoritmos para SHA-3.
- **Dez 2008:** 51 algoritmos selecionados para a **Rodada 1**.
- **Jul 2009:** Redução para **14 algoritmos** na **Rodada 2**.
- **Dez 2010:** NIST anuncia os **5 finalistas** (Rodada 3):
 - BLAKE, Grøstl, JH, Keccak e Skein.
- **Out 2012:** **Keccak** é selecionado como base do SHA-3.
- **Ago 2015:** Publicação oficial do **FIPS 202** (padronização do SHA-3).
- “Padronizado” aqui significa tornar-se um **FIPS** (*Federal Information Processing Standard*) nos EUA.
- Embora o uso obrigatório seja para sistemas governamentais dos EUA, a participação internacional no processo faz com que SHA-3 seja amplamente adotado globalmente, assim como o AES.

Parâmetros de Segurança e SHAKE (XOFs)

- **Comprimentos de saída** suportados por SHA-3:

$$n \in \{224, 256, 384, 512\} \text{ bits}$$

- Pela **paradoxo do aniversário**, a complexidade de ataque por colisão é aproximadamente:

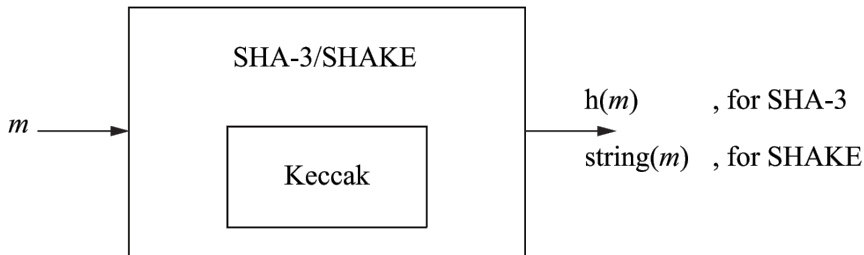
$$2^{112}, 2^{128}, 2^{192}, 2^{256}$$

para saídas de 224, 256, 384 e 512 bits, respectivamente.

- Esses níveis de segurança se alinham com:
 - **AES-128/192/256** (força contra força bruta),
 - **3DES** (força $\approx 2^{112}$).
- SHA-2 também suporta saídas de 224, 256, 384 e 512 bits:
 - Facilita o uso de **SHA-2 ou SHA-3 como alternativas compatíveis**.
- SHA-3 também suporta *extendable-output function* (XOF):
 - **SHAKE128** e **SHAKE256**: produzem saídas de comprimento arbitrário.
 - Oferecem níveis de segurança de aproximadamente 128 e 256 bits.
 - SHA-1 e SHA-2 **não** possuem suporte a XOF.

Visão de Alto Nível de SHA-3 e SHAKE

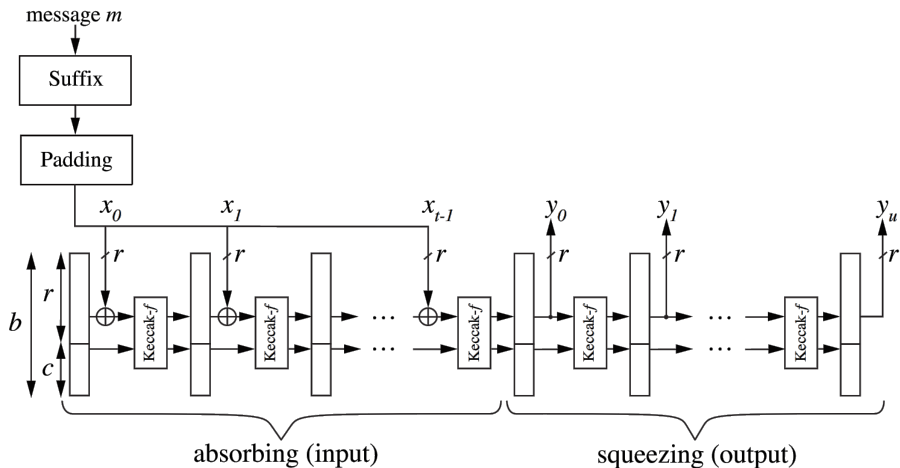
- **SHA-3** e suas variantes **SHAKE128/256** são baseadas no algoritmo **Keccak**.
- Keccak é o **núcleo criptográfico**: uma permutação utilizada no modo esponja.
- A construção **SHA-3/SHAKE** recebe uma mensagem m e:
 - gera um **digest de tamanho fixo** $h(m)$ (caso SHA-3);
 - ou uma **saída de tamanho arbitrário** $\text{string}(m)$ (caso SHAKE).
- Mesma base Keccak, **diferentes parâmetros de uso** (comprimento de saída, segurança efetiva).



Keccak: Construção Esponja (Visão Geral)

- **SHA-3** e as variantes **SHAKE128/256** são baseadas no algoritmo **Keccak**.
- SHA-3:
 - Recebe uma mensagem m .
 - Produz um **digest de tamanho fixo** $h(m)$.
- SHAKE128/256 (*Secure Hash Algorithm Keccak*):
 - São **funções de saída extensível (XOF)**.
 - Geram uma **string de bits de tamanho arbitrário** $\text{string}(m)$.
- Keccak é baseado em uma **construção esponja**:
 - Antes de tudo, há um **preprocessamento** de m (sufixo + padding, divisão em blocos).
 - Depois disso, a esponja possui duas fases:
 - 1 **Fase de absorção**: processa os blocos de entrada x_i .
 - 2 **Fase de espremer (squeezing)**: gera blocos de saída y_j .
- Keccak, por si só, permite gerar **quantos blocos de saída y_j forem necessários**:
 - Para **SHA-3**: apenas y_0 é usado, e seus primeiros bits formam $h(m)$.
 - Para **SHAKE128/256**: usa-se a sequência $y_0, y_1, \dots$ conforme a aplicação.

Keccak: Construção Esponja



Parâmetros da Esponja Keccak: b , r , c

- O “coração” é a função de permutação **Keccak-f**.
- Keccak-f opera sobre um **estado interno** de largura b bits:

$$b = r + c$$

onde:

- b é a **largura do estado** (fixada em $b = 1600$ bits).
- r é a **taxa de bits (bit rate)**:
 - comprimento de cada bloco de mensagem x_i ;
- c é a **capacidade**:
 - controla o **nível de segurança** da construção.
- Diferentes combinações de (r, c) , com $b = 1600$ fixo, resultam em:
 - diferentes **desempenhos** (mais/menos bits absorvidos/expelidos por chamada de Keccak-f);

	function type	b (state) [bits]	r (rate) [bits]	c (capacity) [bits]	security level [bits]	hash output [bits]
SHA3-224	hash	1600	1152	448	112	224
SHA3-256	hash	1600	1088	512	128	256
SHA3-384	hash	1600	832	768	192	384
SHA3-512	hash	1600	576	1024	256	512
SHAKE128	XOF	1600	1344	256	128	arbitrary
SHAKE256	XOF	1600	1088	512	256	arbitrary

SHA-3 / SHAKE: Sufixo e Padding

- Antes do processamento de uma mensagem m , são aplicados:
 - um **sufixo** específico de SHA-3 ou SHAKE;
 - um **padding** definido na especificação de Keccak.
- O sufixo **não** faz parte da função Keccak em si, mas é exigido por SHA-3 e SHAKE128/256:
 - SHA-3: $\text{suf} = 01$;
 - SHAKE128/256: $\text{suf} = 1111$.
- Esses sufixos diferentes garantem **separação de domínio**:
 - a mesma mensagem m não produz a mesma sequência de bits quando usada em SHA-3 e em SHAKE;
 - evita reutilização insegura do mesmo estado interno para funções com propósitos diferentes (hash fixo vs XOF/pseudorrandômico).
- Após anexar o sufixo, aplica-se o **multi-rate padding**:

$$\text{pad}(m, \text{suf}) = m \parallel \text{suf} \parallel 10^*1$$

onde:

- 10^*1 é uma sequência que começa com 1, segue com o menor número possível de zeros e termina com 1;
- o objetivo é que o comprimento total seja um múltiplo de r bits.

SHA-3 / SHAKE: Comprimento da Saída e Uso de y_j

- Após absorver todos os blocos de entrada x_i , Keccak entra na fase de **squeezing**, gerando blocos de saída $y_0, y_1, \dots$, cada um com r bits.
- **SHA-3:**
 - Apenas o **primeiro bloco** y_0 é considerado.
 - y_0 tem r bits, mas o digest requerido é de 224, 256, 384 ou 512 bits.
 - Usa-se apenas os **bits mais significativos** de y_0 ; os demais bits de y_0 são descartados.
- **SHAKE128/256:**
 - Todos os r bits de y_0 podem ser usados.
 - Se a aplicação necessita de mais que r bits:
 - aplica-se a permutação **Keccak-f** novamente ao estado;
 - gera-se $y_1, y_2, \dots$ até alcançar o comprimento desejado.
 - Isso transforma SHAKE em uma **XOF**: função de saída extensível, útil como hash configurável, KDF ou PRNG.
- O tamanho de padding:
 - mínimo: sequência 11 (2 bits);
 - máximo: padrão 10...01 com comprimento $r + 1$ bits.

Função Keccak-f: Permutação Base de SHA-3

- **Keccak-f** é o núcleo de **Keccak**, logo de **SHA-3** e **SHAKE128/256**.
- É uma **permutação** sobre 2^b estados:
 - Cada inteiro de b bits é mapeado *bijetivamente* em outro inteiro de b bits.
- O estado é visto como um arranjo 3D de bits:

$$b = 5 \times 5 \times w$$

- Coordenadas (x, y) formam uma **coluna**, e os w bits ao longo de z são chamados de **lane**.
 - Para SHA-3/SHAKE: $w = 64$ bits é mapeado para: $b = 1600$ bits.
- O estado de 1600 bits encaixa bem em arquiteturas de 64 bits:
 - Pode ser armazenado como um vetor de 25 palavras de 64 bits.

Keccak-f: Tamanhos de Estado e Estrutura de Rodadas

- Keccak-f suporta sete tamanhos de estado:

$$b = 25 \cdot 2^l, \quad l = 0, 1, \dots, 6$$

$$w = 2^l, \quad b = 5 \times 5 \times w$$

- Número de rodadas para cada b :

$$n_r = 12 + 2^l$$

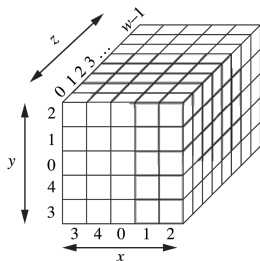
- Para SHA-3/SHAKE128/256: $b = 1600$, $w = 64$, $l = 6$, logo $n_r = 24$.
- Cada rodada de Keccak-f aplica, em sequência, cinco transformações lineares/não lineares:

$$\theta, \rho, \pi, \chi, \iota$$

- Todas as rodadas têm a mesma estrutura.
- Diferem apenas pela **constante de rodada** $RC[i]$, usada na etapa ι .
- Essência: uma permutação altamente misturadora de 1600 bits, aplicada várias vezes, que confere à esponja Keccak suas propriedades de segurança.

Visualização do estado interno

Visualização 3D do estado do Keccak, cada pequeno cubo representa um bit. Para o SHA-3 e o SHAKE128/256, a “profundidade” do arranjo ao longo do eixo z é $w = 64$.

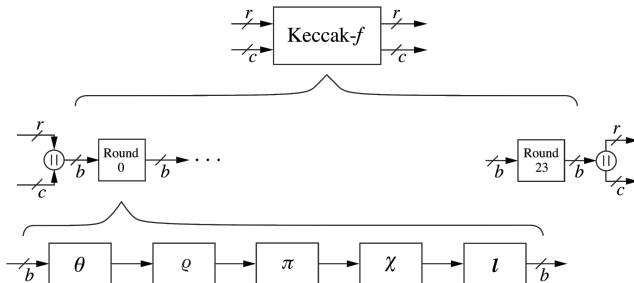


Os diferentes tamanhos de estado e o número de rodadas de Keccak-f; note que $b = 1600$ e $n_r = 24$ para o SHA-3 e o SHAKE128/256.

state b [bits]	25	50	100	200	400	800	1600
number of rounds n_r	12	14	16	18	20	22	24
lane w [bits]	1	2	4	8	16	32	64
l	0	1	2	3	4	5	6

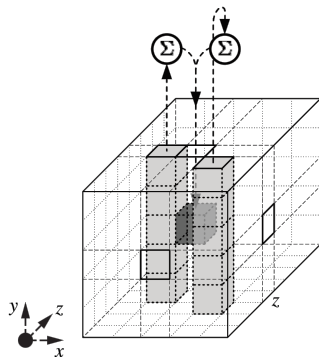
Estrutura Interna da Função Keccak-f

- Cada chamada de Keccak-f aplica $n_r = 24$ rodadas idênticas, e cada rodada é composta em série pelas etapas θ , ρ , π , χ e ι ; apenas a etapa ι recebe uma constante de rodada $RC[i]$ diferente em cada iteração.



Etapa θ do Keccak-f

- Transformação linear de difusão do Keccak-f: cada bit do estado é substituído pelo XOR entre si e com outros 10 bits de sua vizinhança, espalhando a influência de cada coluna por todo o estado.
- Na implementação, calculam-se:
 - $C[x] = A[x, 0] \oplus A[x, 1] \oplus A[x, 3] \oplus A[x, 4]$
 - $D[x] = C[x - 1] \oplus \text{rot}(C[x + 1], 1)$
 - $A'[x, y] = A[x, y] \oplus D[x]$



Etapa ρ do Keccak-f

- O estado é visto como um arranjo 5×5 de *lanes* $A(x, y)$, cada um com w bits (para o SHA3 e SHAKE128/256 $w = 64$).
- Cada lane é submetida a uma **rotação circular de bits** com um deslocamento (offset) específico, que depende exclusivamente das coordenadas (x, y) dessa lane.
- Todos os deslocamentos são tomados módulo w , e foram escolhidos de forma a espalhar os bits ao longo da dimensão z , contribuindo para uma boa difusão em combinação com as demais etapas da permutação.
- A tabela abaixo mostra, para cada posição (x, y) , o número de bits de rotação aplicado na etapa ρ .

	$x = 3$	$x = 4$	$x = 0$	$x = 1$	$x = 2$
$y = 2$	153	231	3	10	171
$y = 1$	55	276	36	300	6
$y = 0$	28	91	0	1	190
$y = 4$	120	78	210	66	253
$y = 3$	21	136	105	45	15

Etapa π do Keccak-f

- A etapa π é uma **permutação das 25 lanes** do estado $A(x, y)$ visto como uma matriz 5×5 .
- Dado o novo arranjo A' , a regra de permutação é:

$$A'[x, y] = A[x + 3y, x], \quad x, y = 0, 1, 2, 3, 4$$

onde todas as coordenadas são computadas módulo 5.

- Nenhum bit é modificado, apenas **reordenado**:
 - a etapa π redistribui as lanes no plano (x, y) ;
 - prepara o estado para que as operações seguintes (especialmente a χ) combinem bits de posições que antes não interagem.
- Exemplo: a posição $A'[2, 3]$ será preenchida pela lane

$$A[2 + 3 \cdot 3, 2] = A[11, 2] = A[1, 2] \quad (\text{mód } 5),$$

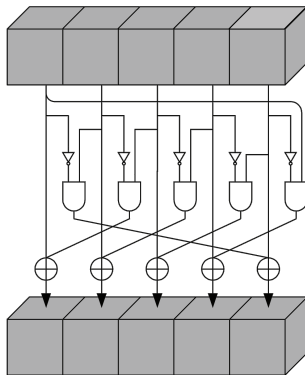
isto é, a lane originalmente em $(1, 2)$ é movida para a borda inferior direita.

Etapa χ do Keccak-f

- **Única operação não linear** dentro do Keccak-f.
- Opera por linhas de lanes: para cada coordenada (x, y) temos

$$A'[x, y] = A[x, y] \oplus (\overline{A[x+1, y]} \wedge A[x+2, y]), \quad x, y = 0, 1, 2, 3, 4$$

onde os índices são tomados módulo 5.



Etapa ι do Keccak-f

- A etapa ι adiciona uma **constante de rodada** de w bits à lane na posição $(0, 0)$ do estado:

$$A'[0, 0] = A[0, 0] \oplus RC[i].$$

- A constante $RC[i]$ depende da rodada i :
 - o número de rodadas n_r varia com o parâmetro b ;
 - para SHA-3 e SHAKE128/256 temos $n_r = 24$ rodadas, logo $RC[0], \dots, RC[23]$.
- Cada $RC[i]$ é, em essência, um vetor quase todo nulo, com bits pseudoaleatórios em posições específicas (por exemplo 1, 2, 3, 7, 15, 31, 63), gerados por um LFSR de grau 8.
- A função da etapa ι é **quebrar simetrias** entre rodadas: ela insere dependência explícita do índice de rodada na permutação Keccak-f, reforçando a segurança contra ataques estruturais.

Lições Aprendidas sobre Funções Hash

- Funções hash são **sem chave** (keyless) e têm muitas aplicações em sistemas de segurança modernos, como **assinaturas digitais** e **MACs**.
- As três propriedades de segurança fundamentais são:
 - **one-wayness** (pré-imagem difícil),
 - **segunda pré-imagem** difícil,
 - **resistência a colisões**.
- Para resistir a ataques de colisão, a saída de uma função hash deve ter, em geral, **256 bits ou mais**.
- **SHA-2** e **SHA-3** são considerados altamente seguros atualmente; não há ataques práticos conhecidos. Já o **SHA-1** é considerado inseguro.
- Além de algoritmos dedicados (como SHA-2 e SHA-3), é possível construir funções hash a partir de **cifras de bloco**.
- **SHA-3** baseia-se em uma **construção esponja**, o que o torna, internamente, bem diferente de SHA-1 e SHA-2.
- Em **hardware**, SHA-3 tende a ser mais eficiente, sendo adequado para aplicações móveis e embarcadas; em **software**, SHA-2 costuma ser mais rápido do que SHA-3.